Computer Architecture - Study Notes

Sample course notes for the RAG chatbot project.
Topics: pipelining, hazards, caches, virtual memory, RISC vs CISC, performance and AMBA buses.

1. Introduction to Computer Architecture

Computer architecture is the design of the parts of a computer system that are visible to the programmer, together with the way those parts are organised in hardware. It is usually split into two levels. The Instruction Set Architecture (ISA) is the contract between software and hardware: it defines the instructions, registers, data types, addressing modes and memory model that a program can use. Examples of ISAs are x86-64, ARMv8 and RISC-V. The microarchitecture is the actual hardware implementation of an ISA, including the pipeline, caches, branch predictor and execution units. Two processors can share the same ISA but have very different microarchitectures; for example, an Intel Core i9 and an AMD Ryzen both run x86-64 code but are built differently inside.

Most modern computers follow the von Neumann model, in which instructions and data are stored in the same memory and travel over the same bus. Because one bus is shared, the processor cannot fetch an instruction and read data in the same cycle. This limitation is called the von Neumann bottleneck. The Harvard architecture solves it by using separate memories and buses for instructions and data, so both can be accessed at the same time. Many microcontrollers, such as the AVR chips used on Arduino boards, use a pure Harvard design. Modern desktop processors use a modified Harvard architecture: they have separate L1 instruction and L1 data caches, but a single unified main memory.

The main building blocks of a CPU are the datapath and the control unit. The datapath contains the register file, the Arithmetic Logic Unit (ALU), multiplexers and the buses that move data between them. The control unit decodes each instruction and generates the control signals that tell the datapath what to do. A control unit can be hardwired, built from fixed logic gates and finite state machines, which is fast, or microprogrammed, where control signals are read from a small internal memory called the control store, which is easier to modify.

2. Instruction Pipelining

Pipelining is a technique that improves processor throughput by overlapping the execution of several instructions, in the same way that a factory assembly line works on several products at once. Each instruction is divided into stages, and a different instruction occupies each stage in every clock cycle. Pipelining does not make a single instruction finish faster; in fact the latency of one instruction may increase slightly because of pipeline register overhead. What improves is throughput, the number of instructions completed per unit of time.

The classic RISC pipeline, used in the MIPS processor, has five stages. IF (Instruction Fetch) reads the next instruction from memory using the program counter. ID (Instruction Decode) decodes the instruction and reads the source registers from the register file. EX (Execute) performs the ALU operation or calculates a memory address. MEM (Memory Access) reads from or writes to data memory for load and store instructions. WB (Write Back) writes the result into the destination register. Pipeline registers, named IF/ID, ID/EX, EX/MEM and MEM/WB, sit between the stages and hold the intermediate values.

In an ideal k-stage pipeline with no stalls, once the pipeline is full one instruction completes every clock cycle. To execute n instructions, a non-pipelined processor needs n x k cycles, while a pipelined one needs k + (n - 1) cycles. For a large number of instructions the ideal speedup approaches k, the number of stages. For example, running 100 instructions on a 5-stage pipeline takes 5 + 99 = 104 cycles instead of 500, a speedup of about 4.8. In practice the speedup is lower because stages are not perfectly balanced, the clock period is set by the slowest stage, and hazards force the pipeline to stall.

3. Pipeline Hazards

A hazard is a situation that prevents the next instruction in the pipeline from executing in its designated clock cycle. There are three types of hazards: structural, data and control.

A structural hazard occurs when two instructions need the same hardware resource in the same cycle. A common example is a processor with a single memory port, where one instruction is fetching while another is doing a load. The usual fix is to duplicate the resource, for example by using separate instruction and data caches.

A data hazard occurs when an instruction depends on the result of an earlier instruction that has not yet finished. The most common kind is Read After Write (RAW). For example, in the sequence ADD R1, R2, R3 followed by SUB R4, R1, R5, the SUB needs R1 before the ADD has written it back. The main solution is forwarding, also called bypassing: the result is sent directly from the EX/MEM or MEM/WB pipeline register to the ALU input of the next instruction, without waiting for write back. Forwarding cannot solve every case. In a load-use hazard, where a load is immediately followed by an instruction that uses the loaded value, the data is only available after the MEM stage, so the pipeline must insert one stall cycle, also called a bubble. Compilers try to reduce such stalls by reordering independent instructions, a technique called instruction scheduling.

A control hazard, or branch hazard, happens because the processor does not know which instruction to fetch next until a branch is resolved. Simple solutions are to stall until the branch outcome is known, or to always predict not-taken and flush the wrong instructions if the branch is taken. Modern processors use dynamic branch prediction. A 2-bit saturating counter predictor changes its prediction only after two consecutive mispredictions, which works well for loops. A Branch Target Buffer (BTB) stores the target address of recently taken branches so that fetching can continue without delay. Deep pipelines make branch mispredictions expensive, because many instructions must be flushed.

4. Memory Hierarchy and Caches

Processor speed has improved much faster than main memory (DRAM) speed, a problem called the memory wall. To hide this gap, computers use a memory hierarchy: small, fast and expensive memories close to the CPU, and large, slow and cheap memories further away. A typical hierarchy is registers, L1 cache, L2 cache, L3 cache, main memory (DRAM) and finally storage (SSD or hard disk). Typical access times are about 1 ns for L1, 3 to 10 ns for L2, 10 to 20 ns for L3 and 50 to 100 ns for DRAM.

Caches work because programs show locality of reference. Temporal locality means that data accessed recently is likely to be accessed again soon, for example a loop counter. Spatial locality means that data near a recently accessed address is likely to be accessed soon, for example the next element of an array. To exploit spatial locality, caches move data in fixed-size blocks called cache lines, usually 64 bytes.

When the CPU finds the requested data in the cache it is a hit; otherwise it is a miss and the block must be fetched from the next level. The hit rate is the fraction of accesses that hit, and the miss rate is 1 minus the hit rate. Misses are classified by the 3C model: compulsory misses happen on the first access to a block, capacity misses happen because the cache is too small to hold all needed data, and conflict misses happen when too many blocks map to the same cache set.

The main performance metric is Average Memory Access Time: AMAT = Hit time + (Miss rate x Miss penalty). For example, if the L1 hit time is 1 ns, the miss rate is 5 percent and the miss penalty is 100 ns, then AMAT = 1 + 0.05 x 100 = 6 ns. Reducing the miss rate to 2 percent gives AMAT = 1 + 0.02 x 100 = 3 ns, which shows how strongly the miss rate affects performance.

5. Cache Organisation

A memory address is divided into three fields when it is looked up in a cache: the tag, the index and the block offset. The offset selects the byte inside the cache line, the index selects the set, and the tag is compared with the stored tag to check whether the line really holds the requested address. Each line also has a valid bit, and in write-back caches a dirty bit.

In a direct-mapped cache each memory block can be placed in exactly one cache line. It is simple and fast, but two frequently used blocks that map to the same line keep evicting each other, causing conflict misses. In a fully associative cache a block can go in any line, which removes conflict misses but requires comparing the tag against every line at once, which is expensive in hardware. A set-associative cache is a compromise: the cache is divided into sets, and each block can go in any of the N lines (ways) of one set. Most L1 caches are 4-way or 8-way set associative.

When a set is full and a new block arrives, a replacement policy chooses which block to evict. LRU (Least Recently Used) evicts the block that has not been used for the longest time; it performs well but is costly to implement exactly for high associativity, so processors often use pseudo-LRU. Other policies are FIFO and random replacement.

Caches also need a write policy. With write-through, every write updates both the cache and the next level of memory; it is simple and keeps memory consistent but generates a lot of traffic, so a write buffer is normally used. With write-back, writes only update the cache and the line is marked dirty; the data is written to memory only when the dirty line is evicted. Write-back reduces memory traffic and is used by most modern L1 and L2 caches. On a write miss, a write-allocate cache loads the block into the cache first, while a no-write-allocate cache writes directly to memory.

6. Virtual Memory

Virtual memory gives each process the illusion of having its own large, private address space. Programs use virtual addresses, and the hardware together with the operating system translates them into physical addresses. This provides isolation between processes, so one program cannot corrupt another, and it allows the total memory used by programs to be larger than the physical RAM.

Memory is divided into fixed-size blocks called pages, typically 4 KB. The operating system keeps a page table for each process that maps virtual page numbers to physical frame numbers. Each page table entry also contains a valid bit and protection bits for read, write and execute permissions. If a program accesses a page that is not in physical memory, a page fault occurs; the operating system then loads the page from disk, which takes millions of cycles, so page faults must be rare.

Looking up the page table in memory on every access would double the memory access time. To avoid this, processors use a Translation Lookaside Buffer (TLB), a small, fast cache of recent virtual-to-physical translations. TLB hit rates are usually above 99 percent. On a TLB miss, the hardware or the operating system walks the page table to find the translation and stores it in the TLB.

7. RISC versus CISC

RISC (Reduced Instruction Set Computer) and CISC (Complex Instruction Set Computer) are two philosophies of ISA design. CISC processors, such as x86, provide many powerful instructions, variable-length instruction encoding and instructions that can operate directly on memory. The goal was to reduce the number of instructions per program, which mattered when memory was expensive.

RISC processors, such as ARM, MIPS and RISC-V, use a small set of simple instructions with a fixed length, usually 32 bits. They follow a load-store architecture: only load and store instructions access memory, and all arithmetic is done on registers. RISC designs normally have many general-purpose registers, for example 32 in RISC-V. Simple, uniform instructions are easy to decode and pipeline, which allows high clock speeds and low power consumption, and is why ARM dominates mobile devices.

The difference has become smaller in practice. Modern x86 processors translate complex CISC instructions internally into simpler RISC-like micro-operations (micro-ops), which are then executed by a pipelined, out-of-order core. RISC-V is an open and free ISA, so anyone can design a RISC-V processor without paying licence fees; it is becoming popular in embedded systems, academia and custom chips.

8. Measuring Performance

The fundamental equation of processor performance is: CPU time = Instruction count x CPI x Clock period, where CPI is the average number of clock cycles per instruction. Equivalently, CPU time = (Instruction count x CPI) / Clock rate. For example, a program with 2 billion instructions, an average CPI of 1.5 and a clock rate of 3 GHz takes (2 x 10^9 x 1.5) / (3 x 10^9) = 1 second. The instruction count depends on the program, compiler and ISA; CPI depends on the microarchitecture; and the clock rate depends on the technology and the pipeline design.

Clock rate alone is a misleading measure, sometimes called the megahertz myth: a 4 GHz processor with a CPI of 2 is slower than a 3 GHz processor with a CPI of 1 running the same instructions. MIPS (millions of instructions per second) is also misleading, because it ignores how much work each instruction does. Real comparisons use benchmark suites such as SPEC CPU.

Amdahl's Law describes the limit of improving only part of a system. If a fraction f of the execution time is sped up by a factor s, the overall speedup is 1 / ((1 - f) + f / s). For example, if 80 percent of a program can be made 10 times faster, the overall speedup is 1 / (0.2 + 0.08) = about 3.57. Even with infinite speedup of that part, the maximum is 1 / 0.2 = 5. The lesson is to make the common case fast and to remember that the part you do not improve eventually dominates. Amdahl's Law is especially important for multicore processors, where the serial part of a program limits parallel speedup.

9. On-Chip Buses: AMBA APB and AHB

In a System-on-Chip (SoC), the processor, memories and peripherals are connected by on-chip buses. ARM's AMBA (Advanced Microcontroller Bus Architecture) is the most widely used family of on-chip bus standards. It includes APB (Advanced Peripheral Bus), AHB (Advanced High-performance Bus) and AXI (Advanced eXtensible Interface).

APB is a simple, low-power bus used for slow peripherals such as UART, timers, GPIO and SPI controllers. It is not pipelined, and every transfer takes at least two clock cycles: a SETUP phase, in which the master asserts PSEL together with the address and control signals, followed by an ACCESS phase, in which PENABLE is asserted. A slave can extend the access phase by holding PREADY low, which inserts wait states. Because of its simplicity, APB is easy to design and verify.

AHB, and its simplified single-master version AHB-Lite, is used for high-performance components such as on-chip memory and DMA controllers. AHB is pipelined: the address phase of one transfer overlaps with the data phase of the previous transfer, so in the best case one transfer completes every cycle. AHB supports burst transfers such as INCR4, INCR8 and wrapping bursts, which move several consecutive words efficiently. The HTRANS signal indicates the transfer type (IDLE, BUSY, NONSEQ or SEQ), and a slave can stall the bus by driving HREADY low. Usually an AHB-to-APB bridge connects the fast AHB system bus to the slower APB peripheral bus. AXI, used in more powerful SoCs, goes further with separate read and write channels and support for multiple outstanding transactions.